pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Course 9 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk head's request

The request

The morning brief on a link I can send.

Say what date its figures are from.

Show the checks it passed.

Refuse anything that asks for a view.

Never hold a key.

A public repo, tests on every change.

A log of every request; a line on the cost.

Fourteen deliverables

deploykit complete; desk_app_main.py

the snapshot, hashed; the app's tests

every check empty; the repo's files

the deploy, on your accounts; a Space (optional)

monitoring; the cost line; a workbook

the description; Git with a pull request

an AI-use log

Shipped for real on the course's accounts: one public repo and one Vercel project, both pfi-desk-app.

The rules for a public app

Allowed names

the brief as of 2026-10-02

the check table

the saved run

Never called

the desk's view

advice or a call

a recommendation

Allowed names

included in Hobby as read

the free tier as read

the platform line

Never called

guaranteed

unlimited

free forever

Describe; date every response; refuse a view first; no key on the server; firm policy first.

How the snapshot works

The problem	A live desk needs a key, a toolkit, and a bill on the server; a snapshot is the desk's output written down once as data
What it keeps	The brief's three sections, the check table with its failures, the run line, the answers kept, the refusal rule as data
Traced	The SHA-256 of every saved exchange the brief came from; a run whose exchange is not saved is refused
You control	Which answers are kept, by a rule you write down, and the as-of date
What comes out	One JSON file, read once at start and never changed, served with no model and no key

The snapshot, rebuilt and hashed

53 / 2

the brief's checks passed and failed, as in Course 8

3

recorded answers kept: passed and sent requests

21 of 21

saved exchanges whose SHA-256 matches

```
desk_snapshot.json: 25,853 bytes, the same bytes as days 8 and 9
```

The brief's date is not the deploy date

Every figure is as of 2026-10-02 in the course's files. The app never says today and never updates a figure.

How to measure what a server installs

The problem	A server installs what a requirements file names; read the code to write it, do not guess
requirements_for	ast over each module: the distribution behind every third-party import, inside functions too
installed_size	Each distribution's own dependency list through the installed metadata, bytes added up: what pip puts on a disk
You control	Which modules are measured; the sizes are the installed versions' own, and Linux wheels differ
What comes out	Two sizes against the platform's bundle limit, and a decision with a number behind it

How the app is built

Routes, wrappers	A route answers a path; a middleware wraps every route, before the request and after the response
The order	A middleware added later runs earlier: the cap added first sits innermost, the log added last sits outermost
Every setting	The snapshot's path, the sink, the two clocks, the cap, the environment: arguments of make_app, platform defaults
You control	The order and the defaults; a notebook and a test pass a fake clock and a list and get the same app
What comes out	One module, one function, one line: app = make_app(), which is what the platform runs

What to deploy, and the app

94

distributions to replay the desk, 923.4 MB here

11

distributions for the snapshot service, 9.6 MB

```
make_app(...):
  desk_app(snapshot, live_settings(...))
  daily_cap          # innermost
  X-Desk-As-Of       # every response
  log_middleware     # outermost
```

```
GET  /health   200 body yes  header 2026-10-02
GET  /brief    200 body yes  header 2026-10-02
GET  /checks   200 body yes  header 2026-10-02
GET  /nowhere  404 body no   header 2026-10-02
POST /ask      404 body no   header 2026-10-02
POST /ask      422 body no   header 2026-10-02
```

A cap of 2: [200, 200, 429], three log lines; the 429 is logged.

A middleware added later runs earlier. On a serverless platform the cap is a floor per process.

Every setting of the app

Function	Setting	Default	What it controls	The course passes
desk_app_main.make_app	snapshot_path	/desk_snapshot.json	the snapshot's JSON file; the default is the file beside the module	the default, here and on the platform
desk_app_main.make_app	sink	to_stdout()	where each log line goes: standard output on a platform, a file or a list here	a list or a file here; to_stdout on the platform
desk_app_main.make_app	clock	None	the counter the log times a request with; None is the real one	fake_clock's counter here; None on the platform
desk_app_main.make_app	wall	None	the UTC clock the log stamps with and the cap counts the day by; None is the real one	fake_clock's wall here; None on the platform
desk_app_main.make_app	daily_cap	1000	the requests one process answers in a UTC day; the next gets 429	DAILY_CAP; 2 in question 4.2
desk_app_main.make_app	environ	None	where the mode is read from; None is os.environ	{} here; None, so os.environ, on the platform
dk.desk_app	snapshot	(required)	the loaded snapshot the routes answer from	json.loads of desk_snapshot.json
dk.desk_app	settings	(required)	live_settings' mode and whether a key loaded, never a value	live_settings(environ)
dk.live_settings	environ	(required)	the mapping DESK_MODE and OPENAI_API_KEY are read from	{} here; os.environ on the platform
dk.daily_cap	app	(required)	the FastAPI app the cap wraps	desk_app's app
dk.daily_cap	max_requests	(required)	the cap for one UTC day in one process	daily_cap
dk.daily_cap	wall	None	the clock the cap reads the day from; None is the real one	wall
dk.log_middleware	app	(required)	the FastAPI app the log wraps	the capped app with its header
dk.log_middleware	sink	(required)	the function given each JSON line	sink
dk.log_middleware	clock	None	the counter for latency_ms; None is time.perf_counter	clock
dk.log_middleware	wall	None	the UTC clock for time_utc; None is the real one	wall
dk.fake_clock	start	(required)	the first reading, UTC, as ISO text	"2026-10-02T14:00:00"
dk.fake_clock	step_ms	(required)	how far the clock moves at each reading, in milliseconds	12.0

18 settings, 9 required, 9 with a default, every default the platform's; the notebook and the tests pass fake_clock, a list, and {}.

How the tests work

No server	TestClient hands a request to the app in the same process and returns the response a browser would see
A fresh app	make_app for each test, with a fake clock, a list as the log's sink, and an empty environment
One promise each	The date on every response, the saved answer, the 404, the refusal, no key, one log line and never the question
You control	The requests and the promises; pytest runs them in a fixed order and reports each by name
What comes out	A file that needs FastAPI, httpx2, and pytest: what a CI runner installs

The app's own tests

13

passed in test_desk_app.py, no network, no key

4

requests for a view, each refused

0 / 0

names out of place in the served brief

```
200  refused: True  rule: opinion words
200  refused: True  rule: an ordering of securities
200  refused: True  rule: opinion words
200  refused: True  rule: a real issuer beside a holding
```

The tests need fastapi, httpx2, and pytest: what the workflow installs. The log never holds a question.

How the checks work

Read the files	A folder about to go public is checked by reading its files; each check reads one kind of thing
public_scan	An environment file at any depth, an email, a key-like string, a large file: first three characters and length, never a value
Platform rules	check_deploy_folder: a pinned requirements file, an entrypoint that assigns app, the bundle limit
Names and YAML	secret_names: every name read from the environment, against .env.example; check_workflow: triggers, versions, pytest
What comes out	Five lists of sentences; the deploy waits until every list is empty

The deploy folder, and its checks

The starter folder

```
public_scan  1  ['desk_settings.json: key_like']

pins_check  0  []

check_deploy_folder  1  ["desk_settings.json, line 4: a k...

secret_names  2  ['DESK_MODE', 'OPENAI_API_KEY']

check_workflow  0  []
```

The planted key removed

```
public_scan  0

pins_check  0

check_deploy_folder  0

secret_names  0

check_workflow  0
```

```
app.py as an import: no entrypoint defines app
requirements.txt ['fastapi==0.143.0']
requirements-test.txt ['fastapi==0.143.0', 'httpx2==2.13.1', 'pytest==9.1.1']
```

```
README pull request:   README.md   8   0
```

The check is stricter than Vercel's page: write app = desk_app_main.app.

How shipping works

Two platforms	Shipping is two platforms reading one repository
A push	The workflow starts a fresh Linux machine per Python: checkout, install, the same pytest command, a run recorded
A pull request	The branch's change as a diff, the checks that ran on it, a merge commit when you say so
Vercel	Detects FastAPI, installs requirements.txt on the Python in .python-version, bundles a function, gives it an address
What comes out	A run history for ci_history, and an address the live-deploy cell checks against the app here

Ship it: GitHub

```
Set up job (34 lines)
Run actions/checkout@v7 (86 lines)
Run actions/setup-python@v7 (12 lines)
Run python -m pip install --upgrade pip (12 lines)
Run python -m pip install -r requirements-test.txt (76 ...
Run python -m pytest -q test_desk_app.py (13 lines)
Post Run actions/setup-python@v7 (1 lines)
Post Run actions/checkout@v7 (32 lines)
Complete job (1 lines)
```

```
Successfully set up CPython (3.13.16)
13 passed in 0.55s
```

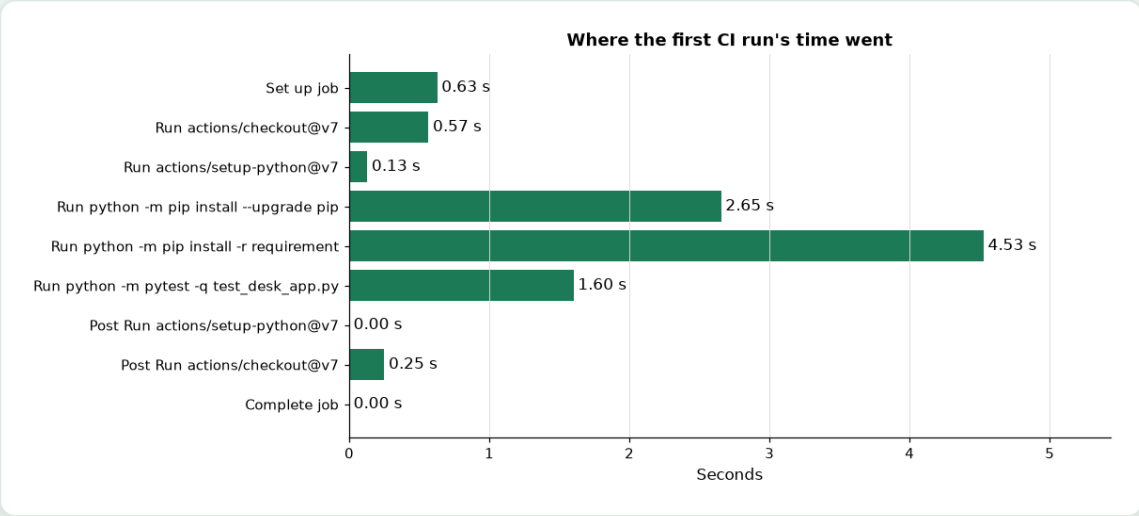
```
test (3.12)   pass   14s
test (3.12)   pass   10s
test (3.13)   pass   15s
test (3.13)   pass   14s
```

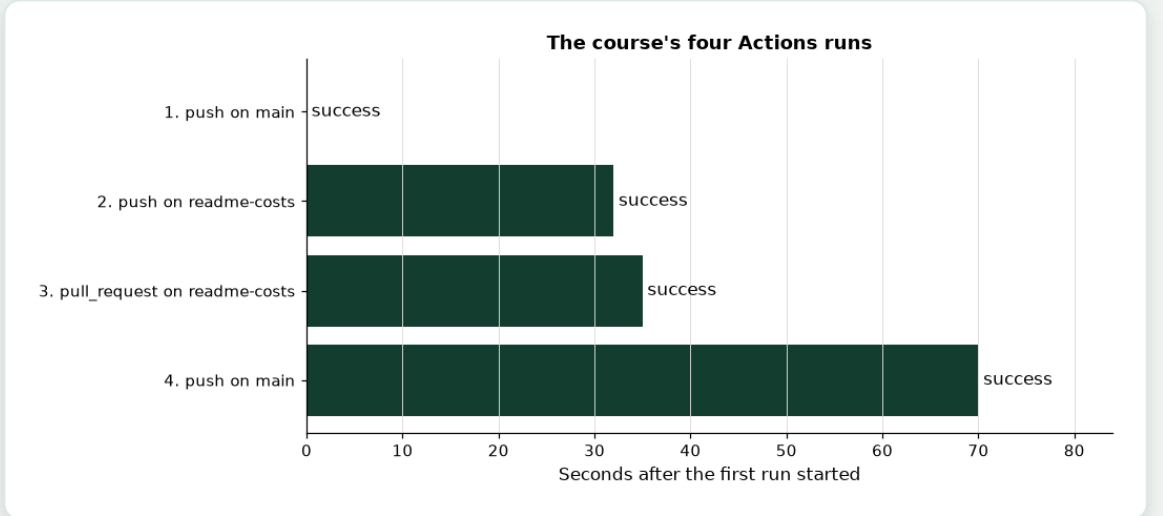
gh pr merge: 1 file changed, 8 insertions(+)

```
runs listed: 4, all success; then one more: the tag's push ran the tests
```

GitHub Actions: free on public repositories, standard runners (docs.github.com, GitHub Actions billing, read 2026-10-12).

The first CI run, and the four runs





One run on one day: yours will differ. All four runs ended in success.

Ship it: Vercel

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.

> vercel deploy --prod --yes

? Created         student/pfi-desk-app

Build Completed in /vercel/output [7s]

? Aliased         https://student.vercel.app

? Ready in 25s
```

```
date 2026-10-12
target vercel production, pfi-desk-app
url_masked https://student.vercel.app
tests_passed 13
status ready
```

```
    request  status      as_of  same_as_here
GET /health     200 2026-10-02          True
 GET /brief     200 2026-10-02          True
GET /checks     200 2026-10-02          True
  POST /ask     200 2026-10-02          True
  POST /ask     404 2026-10-02          True
```

Vercel Hobby: free; non-commercial, personal use only; 100 deployments a day (vercel.com/docs/plans/hobby, read 2026-10-12).

The Space, optional

```
space_folder: 6 files;  check_space_folder: []
```

Why optional

A free account must be older than 30 days, with a verified email, and holds two ZeroGPU Spaces at most.

The required public address is Vercel's, which has no account-age rule.

The course recorded no Space today: day 10 recorded one and deleted it, keeping the slot.

Hugging Face: 2 ZeroGPU Spaces for a free account older than 30 days (huggingface.co/docs/hub/spaces-zerogpu, read 2026-10-12).

How monitoring works

The app's line	The platform's view stops at its edge; log_middleware writes one line per request from inside
Seven columns	UTC time, request id, path, status, latency, bytes out, refused; never a body, a query, a header value, an address
Three readers	read_log makes a table; log_summary counts by path; health_report reads each UTC day against two limits
You control	The limits and the sink; the same readers fit the app's log, the 30-day log, and the platform's lines
What comes out	A day's status by rule, read with the app's deliberate answers in mind

Monitoring: three logs

```
/ask          20          4          0          6    12.0    12.0
/brief        10          0          0          0    12.0    12.0
/checks       10          0          0          0    12.0    12.0
/health       10          0          0          0    12.0    12.0
```

The app's day: alert, error share 0.08; ok without the 404s.

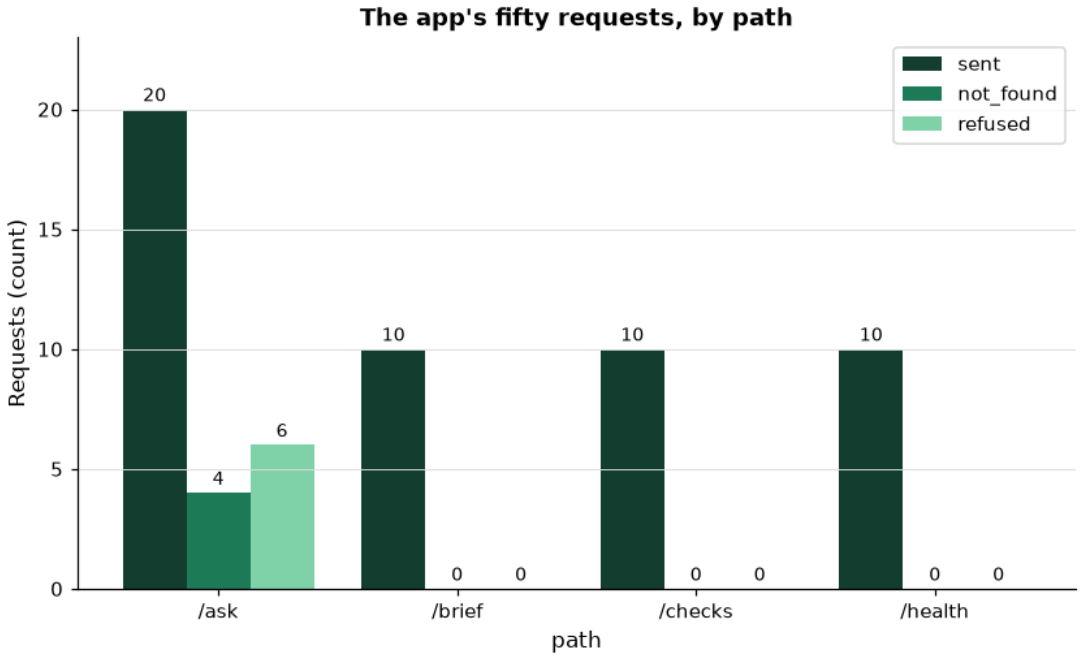
30-day log alerts: 2026-09-17, 2026-09-24

```
13:37:59.34  /ask          404   4.6 ms
13:37:59.10  /ask          200   3.3 ms
13:37:58.88  /checks       200   1.5 ms
13:37:58.61  /brief        200   1.8 ms
13:37:58.30  /health       200   1.2 ms
13:37:56.86  /health       200   3.4 ms
13:37:54.98  /             404   0.8 ms
13:37:54.98  /favicon.ico  404   0.7 ms
13:37:54.97  /favicon.png  404   0.8 ms
```

Three requests nobody sent, all 404. A function's disk is not a log store.

Vercel Hobby: 1 hour of runtime logs; stdout is level info (vercel.com/docs/logs/runtime, read 2026-10-12).

The app's own log

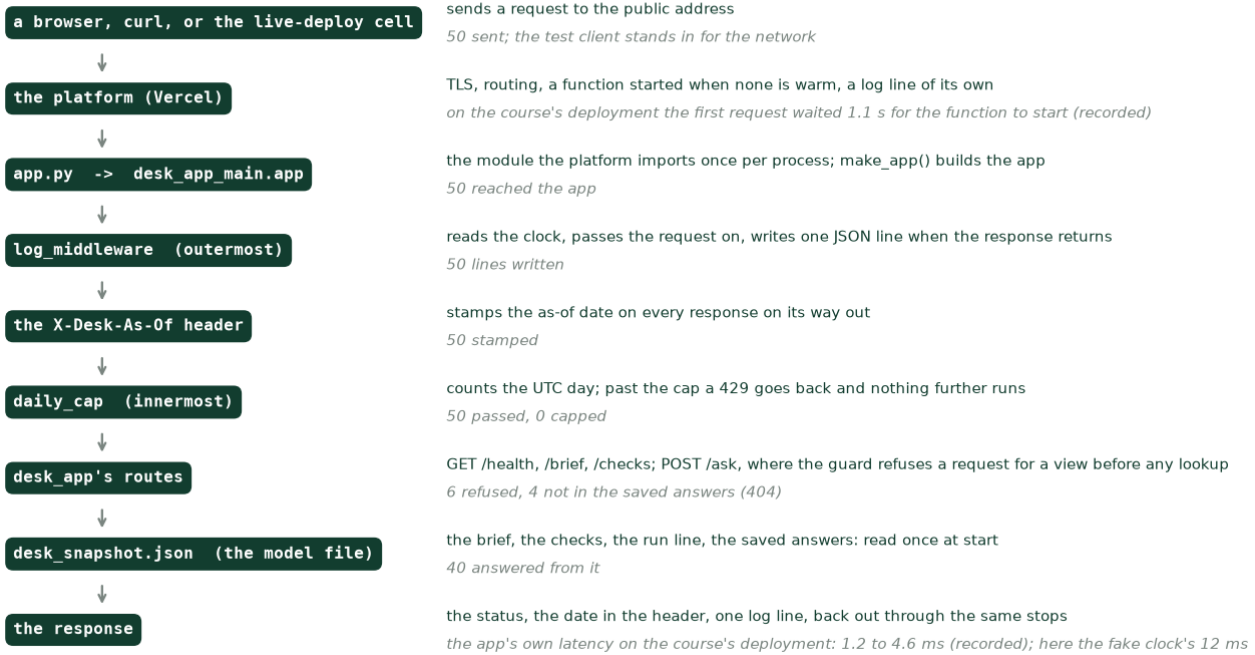


Path	Sent	404s	Refused
/ask	20	4	6
/brief	10	0	0
/checks	10	0	0
/health	10	0	0

The refusals and the 404s are all on /ask; the other paths answered every request.

The path of one request

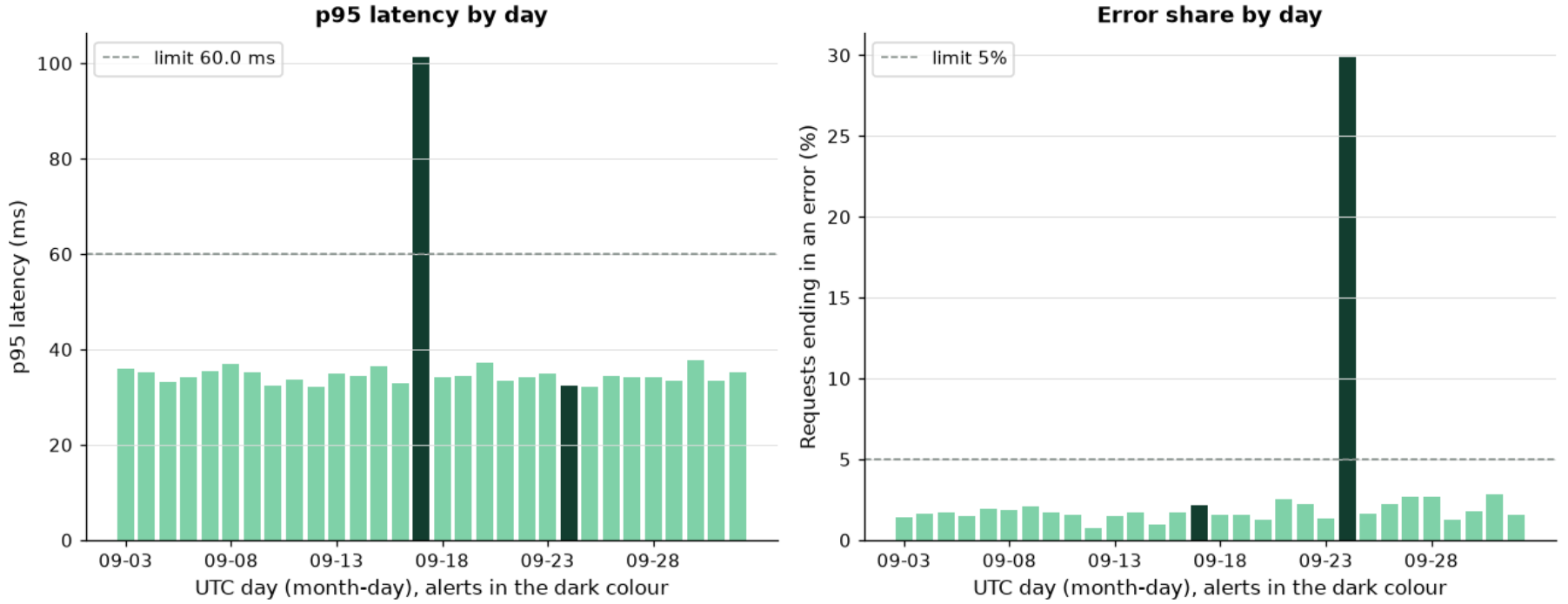
The path of one request through the finished app, and what this run's fifty requests met at each stop



Stop	What the fifty met
a browser, curl, or the liv...	50 sent; the test client stands in for the ...
the platform (Vercel)	the first request waited 1.1 s for the func...
app.py -> desk_app_main.app	50 reached the app
log_middleware (outermost)	50 lines written
the X-Desk-As-Of header	50 stamped
daily_cap (innermost)	50 passed, 0 capped
desk_app's routes	6 refused, 4 not in the saved answers (404)
desk_snapshot.json (the mo...	40 answered from it
the response	the app's own latency on the course's deplo...

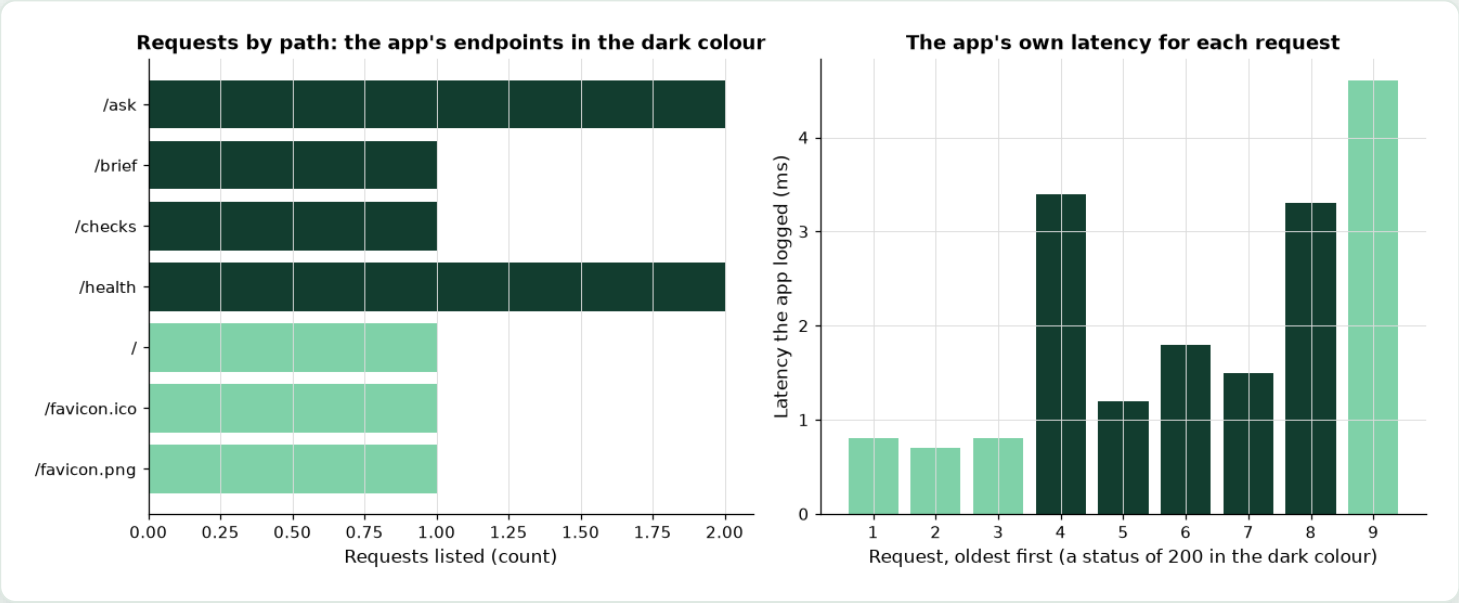
Two figures are not the fake clock's: the wait for the first start and the app's latency, both from the platform's recorded log.

The 30-day log, day by day



The alert days: 2026-09-17 (p95 101.200 ms); 2026-09-24 (p95 32.500 ms).

What the platform listed



Path	Status	Latency (ms)	Refused
/	404	0.8	False
/favicon.ico	404	0.7	False
/favicon.png	404	0.8	False
/health	200	3.4	False
/health	200	1.2	False
/brief	200	1.8	False
/checks	200	1.5	False
/ask	200	3.3	True
/ask	404	4.6	False

Three requests nobody sent, all 404. The platform's text view lists no latency.

How the cost meter works

The problem	A free tier is a set of monthly amounts; a request log is what was used; read the month before the bill does
Platform side	Group by month; sum invocations, bytes out, and latency in the tier's units; within or over each included amount
The model side	Requests on a path times the tokens one live request sends times the price per token; no free tier; zero in replay
You control	The usage per request, the prices, and the included amounts, each read from a page on a stated date
What comes out	One table, a row per month and measure, which the workbook repeats with two formulas

The cost line

```
2026-09       requests 18384.000000 1000000.0       within
2026-09    transfer_gb     0.013379     100.0       within
2026-09 duration_hours     0.074961       4.0       within
2026-09      model_usd     0.000000       NaN no free tier
```

US$0

September in replay: no request to a model

US$47.91

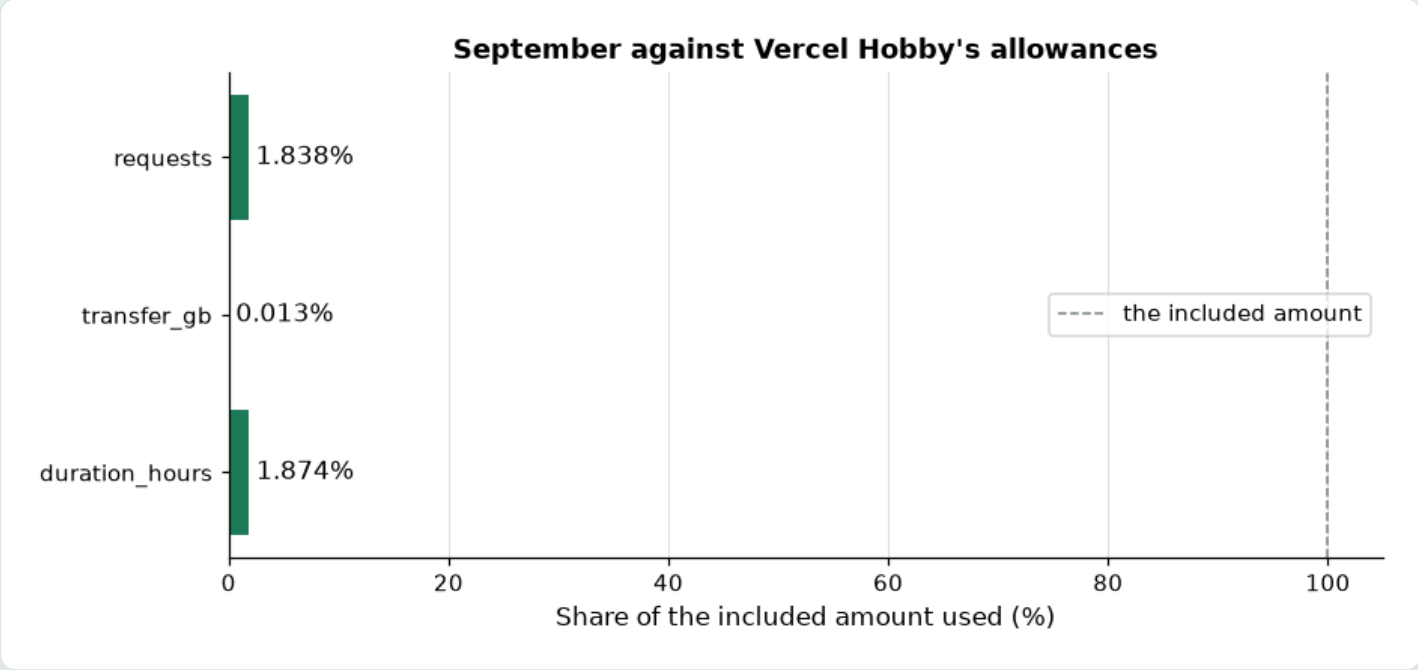
September if the desk went live, at Course 8's prices

4

formulas in the workbook, read back as written

Vercel Hobby: 1,000,000 invocations, 4 CPU hours, 100 GB a month (vercel.com/docs/plans/hobby, read 2026-10-12).

September against Hobby



Measure	Share (%)
requests	1.8384
transfer_gb	0.0134
duration_hours	1.8740

Every bar is a sliver of its dashed line: the app in replay costs nothing at this traffic.

Excel, the description, and Git

Excel side by side (tested in Excel)

`=IF(B2>C2,"over","within")` `free_tier D2:D4`	within, within, within
`=SUMPRODUCT(C2:C5,D2:D5)` `model_cost F2`	47.914342

```
description.md: 436 words; opinion_guard words found: none
8 commits, one per deliverable that writes a file; tag course9
```

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

A key-like string in any text file, of any name, reported.

2

Code back

Written for the course in the style of an AI answer. It runs.

3

Test first

A folder under Git, a tracked settings file with the fake key: assert it is found.

4

Fix

Drop the shortcut: the assistant's version 0; the fixed version 1

Today: a folder holding .git is declared safe and nothing is checked; Git tracks any file it is given.

Code that runs is not the same as code that is right.

What you can do now

- A snapshot whose every figure traces to a saved exchange
- A date on every response, a cap, and a log, in order
- Every check empty before anything goes public
- A public repo with CI, and a pull request read by its diff
- A public app, checked against the one on your machine

Next The Bonus course, Build Your Own Coding Agent